

Kerberos/NFS 개요

`kerberos-nfs`는 FARM/LAB 서버에서 Kerberos로 사용자를 인증하고 NFS 공유 스토리지의 파일 권한을 동일한 UID/GID 기준으로 적용하는 구성을 설명한다.

이 영역을 이해하려면 [설계](#)에서 Kerberos의 주요 개념, NFS 인증 흐름과 FARM/LAB 설정을 확인한다. 실제 설정 변경과 장애 대응은 [운영](#), 과거 장애의 재현 조건과 분석 결과는 [디버깅 로그](#)에서 확인한다.

문서 구성

문서	내용
현재 페이지	Kerberos/NFS 영역의 목적과 문서 구성
설계	Kerberos 핵심 개념, NFS 인증 흐름, FARM/LAB 적용 설정과 선택 이유
운영	계정과 credential 관리, mount 점검, KVNO·NFS 장애 대응
디버깅 로그	재현된 장애의 증상, 실험 조건, 증거와 복구 결과

Kerberos/NFS 설계

[개요](#) · [운영](#) · [디버깅 로그](#)

1. 개요

Kerberos는 비밀번호를 매번 전달하는 대신 중앙 서버가 발급한 암호화된 티켓을 주고받아 신원을 증명하는 인증 시스템이다. `Kerberos/NFS` 구성의 목표는 사용자가 자신의 Kerberos identity로 NFS service에 인증하고, AD에 기록된 UID/GID와 스토리지의 파일 권한에 따라 공유 경로를 사용하는 것이다. Kerberos는 사용자와 service의 identity를 확인하고, NFS는 인증 결과와 numeric UID/GID, mode, ACL을 함께 사용해 파일 접근 권한을 결정한다.

이 문서는 다음 내용을 설명한다.

- Kerberos 인증에 사용되는 principal, ticket, keytab과 ccache
- Kerberos ticket이 NFS의 RPCSEC_GSS 인증으로 이어지는 과정
- FARM과 LAB에 적용한 realm, NFS service principal, protocol과 mount 설정
- 사용자 credential과 NFS service keytab을 관리하는 기준
- FQDN mount source, KVNO와 `sec=krb5`를 선택한 이유

환경별 서버를 처음 구성하거나 현재 설정값을 확인할 때는 다음 가이드를 사용한다.

환경	설정 가이드	현재 기준
FARM	FARM Kerberos 설정 가이드	Samba AD, Synology NAS, NFSv4.0, <code>sec=krb5</code>
LAB	LAB Kerberos 설정 가이드	Samba AD, Linux storage, NFSv4.1, <code>sec=krb5</code>

2. 핵심 개념

2.1 Kerberos 개념

용어	의미	현재 구성에서의 사용
Realm	하나의 Kerberos 인증 관리 영역	<code>FARM.DECS.INTERNAL</code> , <code>LAB.DECS.INTERNAL</code>
Principal	Kerberos가 식별하는 사용자 또는 service 이름	<code><username>@REALM</code> , <code>nfs/<fqdn>@REALM</code>
KDC	principal의 비밀키를 관리하고 ticket을 발급하는 서버	FARM/LAB의 Samba AD DC가 KDC 역할을 함께 수행
TGT	사용자가 다른 service ticket을 요청할 때 사용하는 ticket	host의 사용자 ccache에 저장
Service ticket	특정 service에 접속하기 위해 KDC가 발급하는 ticket	NFS 접근 시 <code>nfs/<storage-fqdn>@REALM</code> 용 ticket 발급
SPN	service를 나타내는 principal 이름	NFS server의 FQDN을 포함한 <code>nfs/<fqdn></code> 등록
Keytab	principal의 장기 비밀키를 파일로 저장한 credential	사용자 keytab은 계산 host root가 보관하고 NFS service keytab은 storage가 보관
Ccache	발급된 TGT와 service ticket을 저장하는 credential cache	<code>/run/user/<uid>/krb5cc</code> 를 사용자 process가 사용
KVNO	principal 비밀키가 바뀔 때마다 올라가는 version 번호. 티켓을 만들 때 쓴 키 버전과 그걸 확인하는 keytab의 키 버전이 다르면 인증이 실패함.	AD의 NFS SPN과 storage keytab의 key version 일치 여부 확인
RPCSEC_GSS	Kerberos credential을 NFS RPC 인증에 사용하는 방식	NFS mount의 <code>sec=krb5</code> 가 GSS security context를 사용
RFC2307	AD에 Unix UID/GID를 저장하는 속성 체계	사용자·group을 스토리지와 container에서 같은 숫자로 표현

2.2 NFS 파일권한 개념

Kerberos가 다루지 않는, NFS 접근 결정에 함께 쓰이는 파일시스템 고유 개념이다.

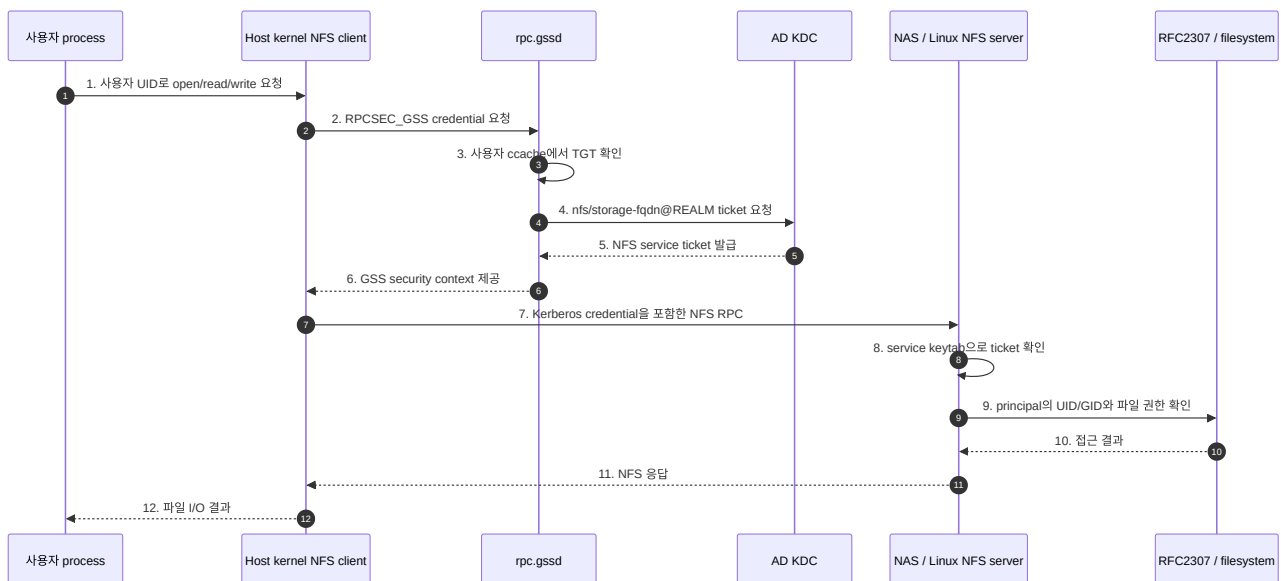
용어	의미
UID/GID	사용자·그룹을 식별하는 숫자. Linux와 NFS는 이름이 아니라 이 숫자로 파일 권한을 판단한다

용어	의미
Identity	이 문서에서 "누가 누구인가"를 나타내는 정보 전체. Kerberos 쪽에서는 principal, 파일시스템 쪽에서는 UID/GID로 표현되며, 같은 사람을 가리키도록 맞추는 것이 이 설계의 핵심이다
Owner/Group	파일에 지정된 소유 사용자와 소유 그룹(둘 다 UID/GID로 저장됨)
Mode	소유자·그룹·기타 사용자별 읽기/쓰기/실행 권한을 나타내는 Unix 표준 권한 비트(예: 0600)
ACL	mode의 소유자/그룹/기타 3단계보다 세밀하게, 특정 사용자·그룹 여러 개에 개별 권한을 지정할 수 있는 확장 권한 체계

Kerberos 인증과 filesystem 권한 판정은 이어지는 두 단계다. Kerberos는 요청자가 어떤 principal인지 확인하고, NFS server는 그 principal에 대응하는 UID/GID와 파일의 owner, group, mode, ACL을 비교해 접근 결과를 결정한다.

3. Kerberos가 NFS 접근에 적용되는 과정

사용자 credential은 계산 host에서 준비되고, 실제 파일 접근은 host kernel NFS client와 `rpc.gssd`가 처리한다.



각 단계는 다음과 같다.

1. 사용자 process가 UID로 파일 open/read/write를 요청한다.
2. Host kernel NFS client가 `rpc.gssd`에 RPCSEC_GSS credential을 요청한다.
3. `rpc.gssd`가 사용자 ccache에서 TGT를 확인한다.
4. `rpc.gssd`가 TGT로 KDC에 `nfs/storage-fqdn@REALM` service ticket을 요청한다.
5. KDC가 NFS service ticket을 발급한다.
6. `rpc.gssd`가 발급받은 ticket으로 GSS security context를 만들어 kernel에 전달한다.
7. Kernel이 이 credential을 포함한 NFS RPC를 NFS server에 보낸다.
8. NFS server가 자신의 service keytab으로 ticket을 검증한다.

9. NFS server가 ticket의 principal을 RFC2307로 UID/GID로 변환해 파일의 owner, group, mode, ACL과 비교한다.
10. 비교 결과(허용/거부)를 NFS server에 반환한다.
11. NFS server가 결과를 kernel에 응답한다.
12. Kernel이 결과를 사용자 process에 반환한다.

이 흐름에서 사용하는 credential은 세 종류다.

Credential	보관 위치	역할
사용자 keytab	계산 host <code>/etc/decs-krb/keytabs/<username>.keytab</code>	사용자 TGT를 새로 발급
사용자 ccache	계산 host <code>/run/user/<uid>/krb5cc</code>	NFS service ticket을 요청하고 사용자 process의 NFS 접근에 사용
NFS service keytab	Synology NAS 또는 Linux storage	NFS server가 받은 service ticket을 확인

사용자 process에는 `KRB5CCNAME=FILE:/run/user/<uid>/krb5cc` 를 전달한다. Host의 refresh timer가 사용자 keytab으로 ccache를 갱신하며, container는 ccache와 읽기 전용 `/etc/krb5.conf` 를 사용한다.

4. FARM과 LAB에 적용한 설정

FARM과 LAB은 같은 credential 모델과 UID/GID 기준을 사용한다. Storage 구현과 검증된 NFS protocol version에 맞춰 realm, endpoint, service principal과 mount option을 환경별로 관리한다.

4.1 공통 설정 기준

항목	설정 기준
사용자 identity	DB, AD RFC2307, NFS owner와 container에 같은 UID/GID 적용
사용자 keytab	계산 host root 소유, mode <code>0400</code>
사용자 ccache	<code>/run/user/<uid>/krb5cc</code> , 사용자 UID/GID 소유, mode <code>0600</code>
NFS mount source	NFS service principal과 같은 FQDN 사용
NFS 인증	기본 <code>sec=krb5</code>
Client GSS	<code>rpc.gssd</code> 가 호출 UID의 ccache로 GSS context 생성
Storage identity	AD RFC2307 값을 winbind 또는 SSSD/idmap으로 numeric UID/GID에 연결

4.2 환경별 설정값

항목	FARM	LAB
Kerberos realm	FARM.DECS.INTERNAL	LAB.DECS.INTERNAL
DNS domain	farm.decs.internal	lab.decs.internal
AD/KDC	dc1.farm.decs.internal	dc1.lab.decs.internal
Storage 구현	FARM AD에 가입한 Synology NAS	LAB AD에 가입한 Linux NFS server
NFS data 주소	100.100.100.120	100.100.100.100
Mount source	nas.farm.decs.internal:/volume1/share	lab-storage.lab.decs.internal:/294t/dccloud/share
NFS version	vers=4.0	vers=4.1
Security flavor	sec=krb5	sec=krb5
NFS service principal	nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL	nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
Service keytab	/etc/nfs/krb5.keytab	/etc/krb5.keytab
Server GSS process	Synology svcgssd	Linux rpc.svcgssd 또는 NFS server가 관리하는 service
Identity 연결	Samba winbind RFC2307	Storage는 winbind RFC2307, 일반 client는 SSSD RFC2307

4.3 NFS version 선택

FARM의 Synology NAS는 현재 `vers=4.0,sec=krb5,hard,timeo=600,retrans=2` 를 운영 기준으로 사용한다. NFSv4.1 과 RPCSEC_GSS 조합에서 확인된 timeout과 access denied 결과를 반영해 NFSv4.0을 선택했다. `rsize/wsize` 는 1 MiB를 요청하며 실제 연결값은 Synology와의 협상 결과에 따라 128 KiB로 표시될 수 있다.

LAB의 Linux storage는 `vers=4.1,sec=krb5` 를 사용한다. Linux NFS server와 client의 session 동작을 활용하며, session slot 관련 분석은 [NFSv4.1 session slot 고착](#)에서 관리한다.

5. 사용자 credential 관리

5.1 Keytab과 ccache { #keytab-ccache }

구분	Keytab	Ccache
저장 내용	principal의 장기 비밀키	발급된 TGT와 service ticket
용도	새 ticket 발급	ticket 제시와 service ticket 요청
수명	principal key rotation까지 유지	ticket lifetime과 renew lifetime 적용
저장 위치	host root-only 경로	/run/user/<uid>/krb5cc

구분	Keytab	Ccache
사용 주체	host credential refresh service	사용자 process와 <code>rpc.gssd</code>

Keytab은 새 ticket을 계속 발급할 수 있는 장기 credential이다. Host root가 keytab을 관리하고 사용자 runtime에는 수명이 제한된 ccache를 제공해 credential 노출 범위를 줄인다.

5.2 Ccache 갱신

`decs-krb-refresh@<username>.timer` 는 사용자별 oneshot service를 주기적으로 실행한다. 현재 일반적인 갱신 주기는 1시간이다.

유효 ccache 확인

- > renew 기간이 충분하면 임시 ccache에서 `kinit -R`
- > renew 기간이 짧거나 갱신이 실패하면 keytab으로 새 TGT 발급
- > `klist`로 결과 확인
- > `uid:gid, mode 0600` 적용
- > `/run/user/<uid>/krb5cc`로 원자 교체

AD group이 변경된 사용자는 keytab으로 새 TGT를 발급해 최신 group 정보를 ccache에 반영한다.

6. NFS service identity

NFS 서비스(NAS / Linux NFS server) 쪽도 Kerberos principal로 자신을 증명해야 한다. 이 절은 그 principal을 구성하는 이름(FQDN/SPN)과 비밀키 버전(KVNO)을 FARM/LAB이 어떻게 관리하는지 설명한다.

6.1 FQDN과 service principal

Kerberos NFS service는 host-based principal을 사용한다(FQDN은 호스트 이름과 도메인을 모두 포함한 완전한 이름을 뜻한다). Mount source의 hostname과 NFS service principal의 hostname을 같은 값으로 유지한다.

```
FARM mount source: nas.farm.decs.internal:/volume1/share
FARM service SPN:  nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
FARM transport:    addr=100.100.100.120

LAB mount source:  lab-storage.lab.decs.internal:/294t/dcloud/share
LAB service SPN:   nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
LAB transport:     100.100.100.100
```

FQDN은 KDC가 발급할 NFS service ticket을 결정하고 data 주소는 실제 packet 전송 경로를 결정한다. 이 구분을 통해 service identity를 유지하면서 storage network를 사용한다.

6.2 KVNO와 service keytab

KDC가 발급한 service ticket의 KVNO와 storage keytab에 들어 있는 NFS principal의 KVNO가 일치해야 storage가 ticket을 확인할 수 있다.

FARM은 전용 AD service account `svc-nfs-farm`에 NFS SPN을 등록한다. Synology machine account password lifecycle과 NFS service key lifecycle을 분리해 관리자가 명시적으로 rotation할 때 KVNO를 변경한다. Synology keytab에는 FARM과 기존 AILAB NFS principal이 함께 있으므로 rotation 과정에서 두 acceptor를 모두 유지한다.

LAB은 Linux storage computer account `LAB-STORAGE$`의 NFS SPN과 `/etc/krb5.keytab`을 사용한다. 환경별 checker는 AD의 KVNO와 storage keytab의 KVNO를 비교해 drift를 확인한다.

7. NFS security flavor와 권한

Mount option	제공하는 보호
<code>sec=krb5</code>	사용자와 NFS service 인증
<code>sec=krb5i</code>	인증과 RPC payload 무결성
<code>sec=krb5p</code>	인증, 무결성과 RPC payload 암호화

현재 FARM/LAB 기본값은 `sec=krb5`다. 내부 storage network에서 Kerberos identity를 확인하고 payload wrapping 비용을 줄이는 설정이다. 무결성 또는 암호화가 필요한 공유 경로는 성능을 측정한 뒤 `krb5i` 또는 `krb5p`를 선택할 수 있다.

Kerberos 인증이 완료된 뒤에는 numeric UID/GID, file mode와 ACL이 최종 권한을 결정한다. 다음 값은 한 사용자에게 대해 같은 숫자를 유지한다.

AD RFC2307 uidNumber/gidNumber
= NFS storage의 파일 owner/group
= 계산 host가 조회한 UID/GID
= container process의 UID/GID

8. 설정 확인 위치

설정	기준 문서 또는 파일
FARM canonical runbook	kerberos-nfs/docs/farm.md
LAB canonical runbook	kerberos-nfs/docs/lab.md
계산 host mount와 readiness 설정	kerberos_nfs_client_recovery.yml
Monitoring 기대값	exporters.yml

Kerberos/NFS 운영

이 문서는 현재 Docker 기반 FARM/LAB 환경의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

1. 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
2. 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
3. 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
4. mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.
5. `findmnt`, `klist`, `kvno`, readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
6. UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성 (추후 자동화 시스템으로 대체될 예정)

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run` 으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab` 을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber` 가 선택한 DB UID/GID와 일치한다.

- target host keytab이 `root:root 0400` 이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 새로 만든 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf` 만 전달된다.

구현은 `create_container.py`와 `Kerberos command builder`에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \
    /etc/decs-krb/keytabs/<username>.keytab \
    /etc/decs-krb/refresh.d/<username>.env

sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400
refresh env: root:root 600
ccache dir:  <uid>:<gid> 700
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager

sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \
    klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하도록 구현되어 있다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc
sudo systemctl start 'decs-krb-refresh@<username>.service'
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |
  grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)= '

docker exec --user <username> <container> \
  sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름
- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

환경마다 source, NFS version과 service principal이 다르다.

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4
defaults,vers=4.0,rsz=1048576,wsz=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,
_netdev,exec,nouser 0 0
```

LAB의 올바른 형태는 다음과 같다. 실제 fstab에는 rpc-gssd 와 decs-kerberos-nfs-ready.service 에 대한 x-systemd.requires/after option도 playbook이 함께 추가한다.

```
lab-storage.lab.decs.internal:/294t/dcloud/share /home/tako9/share nfs4
defaults,vers=4.1,sec=krb5,proto=tcp,hard,_netdev,exec,nouser 0 0
```

핵심은 source가 각 realm의 NFS service principal과 일치하는 FQDN이라는 점이다.

FARM 올바른: `nas.farm.decs.internal:/volume1/share`
FARM 잘못된: `100.100.100.120:/volume1/share` 또는 `192.168.2.30:/volume1/share`
FARM 정상: runtime option의 `addr=100.100.100.120`

LAB 올바른: `lab-storage.lab.decs.internal:/294t/dcloud/share`
LAB 잘못된: `100.100.100.100:/294t/dcloud/share` 또는 `192.168.1.20:/294t/dcloud/share`
LAB 정상: FQDN을 해석한 실제 transport 주소 `100.100.100.100`

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal 또는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` 과 이름이 맞지 않는다. `192.168.2.30` 과 `192.168.1.20` 은 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
MOUNT_TARGET=/home/tako9/share
STORAGE_FQDN=lab-storage.lab.decs.internal

findmnt -T "$MOUNT_TARGET" -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n "\\|${MOUNT_TARGET}|,+4p"
getent hosts "$STORAGE_FQDN"
```

FARM host에서는 값을 `/home/tako<번호>/share` 와 `nas.farm.decs.internal` 로 바꾼다. `findmnt` 의 `vers=4.0` 은 FARM, `vers=4.1` 은 LAB이어야 하고 두 환경 모두 `sec=krb5` 여야 한다.

`fstab` 설정은 [server-state playbook](#)에서 관리한다. 이미 mount된 legacy superblock은 유지보수 창에 `clean unmount/remount`한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd` 가 필요하다. realm과 NFS principal을 host 이름에 맞춰 선택한다.

```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')

case "$short" in
    FARM*) KRB_REALM=FARM.DECS.INTERNAL
           KRB_NFS_PRINCIPAL=nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL ;;
    LAB*)  KRB_REALM=LAB.DECS.INTERNAL
           KRB_NFS_PRINCIPAL=nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL ;;
    *)     printf '지원하지 않는 host: %s\n' "$short" >&2; exit 1 ;;
esac
KRB_MACHINE_PRINCIPAL="${short}\${KRB_REALM}"

sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "$KRB_MACHINE_PRINCIPAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno "$KRB_NFS_PRINCIPAL"
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd` 를 `-n` 으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n` 은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. Synology NAS와 Linux storage 운영 차이

FARM의 Synology NAS와 LAB의 Linux storage는 같은 NFS/Kerberos 개념을 사용하지만 설정을 관리하는 방식, service 이름과 keytab lifecycle이 다르다. client에서 확인하는 `findmnt`, `klist`, `kvno` 절차는 비슷해도 server 변경 명령은 서로 바뀌 실행하면 안 된다.

7.1 운영 경계 비교

항목	FARM Synology NAS	LAB Linux storage
관리 접속	<code>jy@192.168.2.30:6954</code>	<code>jy@192.168.1.20:6953</code>
AD join 도구	DSM UI 또는 <code>/usr/syno/sbin/synowin</code>	표준 Samba <code>net ads join</code>
Samba/winbind	Synology SMB package 경로와 <code>pkg-synosamba-*</code> unit	<code>/etc/samba/smb.conf</code> , 표준 <code>winbind.service</code>

항목	FARM Synology NAS	LAB Linux storage
RFC2307 NSS	<code>files winbind syno</code> 순서와 Synology idmap 설정을 함께 확인	<code>files systemd winbind</code> 와 Samba <code>idmap config</code> LAB : <code>backend = ad</code> 확인
NFS export root	<code>/volume1/share</code>	<code>/294t/dcloud/share</code> ; PoC는 별도 <code>test_krb</code> export
production export	storage IP별 <code>root_squash</code> , <code>sec=krb5:krb5i:krb5p</code> ; Synology <code>anonuid/anongid</code> , <code>insecure</code> option 포함	Linux <code>/etc/exports</code> 에서 <code>root_squash</code> , <code>sec=krb5</code> , <code>no_subtree_check</code> 를 명시
NFS service	Synology <code>/usr/sbin/svcgssd</code> 와 <code>/usr/sbin/idmapd</code>	<code>nfs-server</code> 와 <code>rpc-svcgssd</code> 또는 배포판의 socket-activated GSS service
NFS keytab	<code>/etc/nfs/krb5.keytab</code> 이 ticket 검증 기준; <code>/etc/krb5.keytab</code> 도 동기화 검사	<code>/etc/krb5.keytab</code> 한 경로
NFS SPN 등록 계정	전용 user <code>svc-nfs-farm</code> ; <code>NAS\$</code> 와 분리	computer account <code>LAB-STORAGE\$</code>
추가 acceptor	같은 keytab의 AILAB <code>nfs/nas.ai lab.dgu@AILAB.DGU</code> 를 반드시 보존	현재 없음
운영 NFS version	v4.0	v4.1
keytab checker	<code>--profile farm</code> , 5분 user timer 운영	<code>--profile lab</code> ; profile은 준비됐지만 PoC 단계에서는 timer를 기본 활성화하지 않음

Synology에서 `getent` , `id` 또는 `wbinfo` 가 DECS UID/GID 대신 `96470xxx` 같은 내부 ID를 반환하면 RFC2307 경로가 정상인 상태가 아니다. 이때 symbolic `chown -R 'FARM\user':'FARM\group'` 을 실행하면 잘못 해석된 숫자가 filesystem에 저장된다. 먼저 DB-AD와 NAS의 numeric UID/GID가 일치하는지 확인하고, ownership 복구도 검증된 숫자로 수행한다.

FARM NAS에는 관리·호환성 목적으로 남은 `192.168.2.0/24 sec=sys,no_root_squash` export가 있을 수 있다. 이것은 production Kerberos data path가 아니며 FARM host나 container가 이 `sec=sys` export를 사용하도록 변경하면 안 된다.

7.2 읽기 전용 점검

FARM Synology NAS에서는 vendor 경로와 service 이름을 사용한다.

```
ssh -p 6954 jy@192.168.2.30

/usr/syno/sbin/synowin -getWorkgroup
SMB=/usr/local/packages/@appstore/SMBService/usr/bin
sudo "$SMB/net" ads testjoin
sudo "$SMB/wbinfo" --online-status

sudo klist -kte /etc/nfs/krb5.keytab
sudo klist -kte /etc/krb5.keytab
sudo exportfs -v | sed -n '/\/volume1\/share/,+8p'
ps -ef | grep -E 'svcgssd|idmapd' | grep -v grep
```

LAB Linux storage에서는 표준 Samba/NFS systemd unit과 keytab을 확인한다.

```
ssh -p 6953 jy@192.168.1.20

sudo net ads testjoin
wbinfo --online-status
getent passwd '<username>'

sudo klist -kte /etc/krb5.keytab \
  | grep -F 'nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL'
systemctl is-active winbind nfs-server
systemctl --no-pager --full status rpc-svcgssd.service 2>/dev/null || true
sudo exportfs -v | sed -n '/\/294t\/dcloud\/share/,+8p'
```

LAB 배포판에서는 `rpc-svcgssd` 가 독립 unit이 아니라 socket 또는 `nfs-server` 의 일부로 관리될 수 있다. unit 이름 하나만으로 실패를 판정하지 말고 process, service keytab과 실제 `kvno` /canary 결과를 함께 본다.

7.3 변경 작업에서 지킬 차이

- Synology 설정을 바꾸기 전에는 DSM/Samba/NFS 설정과 두 keytab을 먼저 backup한다. DSM 또는 SMB package가 관리하는 파일·unit을 일반 Ubuntu와 같다고 가정하지 않는다.
- FARM keytab repair는 AILAB principal과 이전 FARM KVNO를 보존하는 전용 `repair-farm-nas-nfs-keytab.sh` 을 사용한다. `svcgssd` 를 FARM principal 하나로 제한하는 `-p` 나 nameless acceptor `-n` 으로 시작하지 않는다.
- LAB storage는 표준 Samba join, `/etc/krb5.keytab` , `nfs-server` / `rpc-svcgssd` 와 Ansible desired state를 기준으로 한다. FARM repair/rotation script를 LAB에 실행하지 않는다.
- 두 환경 모두 identity/export 변경 후 `exportfs -ra` 와 필요한 NFS/RPC cache flush를 수행할 수 있지만, 먼저 활성 client와 D-state를 확인하고 유지보수 시간에만 실행한다.
- Synology 관리 IP와 LAB storage 관리 IP는 NFS source가 아니다. 변경 후 client `findmnt` 에서 각각 `nas.farm.decs.internal` 과 `lab-storage.lab.decs.internal` 이 유지되는지 확인한다.

환경별 keytab 상태는 같은 checker를 profile만 바꿔 읽기 전용으로 확인한다.

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile lab
```

server-side 전체 설정 절차는 [FARM canonical runbook](#)과 [LAB canonical runbook](#)을 각각 따른다.

8. FARM NAS service account와 KVNO 운영

현재 정책

- NFS SPN 등록 계정: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'
```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재
- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 [FARM config](#)에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair` 는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd` 를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

9. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

10. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME` , ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

11. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt` , `klist` , checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

12. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.

Kerberos/NFS 디버깅 로그

이 문서는 Kerberos/NFS에서 발생한 장애와 재현 실험을 축적하는 인덱스다. 정상 상태를 만드는 절차는 [운영 문서](#)에 두고, 여기에는 장애가 어떻게 관측됐고 어떤 가설을 어떤 조건에서 검증했는지 기록한다.

기록 목록

검증일	상태	문제	핵심 결론
2026-07-23	영구 조치 적용 · 운영 검증 완료	NFSv4.1 session slot 고착 해결 및 운영 적용	LAB storage를 6.12.0-250.el10으로 교체하고 LAB8 운영 mount의 15초 idmap 지연 시험에서도 영구 slot 고착이 없음을 확인함
2026-07-20	관측, 인과 미확정	LAB9에서 sec=krb5를 사용하는 이유	krb5p에서 krb5로 전환한 뒤 D-state 지속 시간, SUNRPC 적체와 mount probe 지연이 크게 감소함
2026-07-16	재현 완료, 과거 장애 원인 후보	NFSv4.1 session slot 고착	구형 storage kernel에서 지연된 idmap decode가 NFSv4.1 slot 하나를 영구 INUSE 상태로 남기는 경로를 재현함

상태는 다음 의미로 사용한다.

- **관측**: 증상과 증거는 있지만 원인 경로를 재현하지 못했다.
- **재현 완료**: 통제된 조건에서 같은 kernel 또는 protocol 경로를 다시 만들었다.
- **원인 후보**: 재현된 경로가 과거 장애를 설명하지만 장애 발생 시점의 직접 trace가 없어 동일 원인이었다고 확정할 수 없다.
- **원인 확정**: 장애가 시작된 시점의 증거와 재현 결과가 같은 경로를 가리킨다.

장애 발생 시 먼저 할 일

NFS **hard** mount 관련 process가 D-state에 들어간 경우에는 새로운 **find**, **du**, **stat**, canary I/O를 반복하지 않는다. 각 명령이 새로운 NFS request와 D-state process를 추가할 수 있다.

먼저 NFS 경로를 읽지 않는 local 상태만 수집한다.

```
date -Ins
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS

ps -e -o state=,pid=,ppid=,etimes=,comm=,wchan:48=,args= \
| awk '$1 ~ /^D/ {print}'

cat /proc/net/rpc/nfs 2>/dev/null || true
cat /run/decs-nfs-forensics/status.json 2>/dev/null || true
cat /var/lib/decs-nfs-gss/recovery.state 2>/dev/null || true
cat /var/lib/decs-nfs-gss/canary.state 2>/dev/null || true

systemctl --no-pager --full status rpc-gssd.service
journalctl -u rpc-gssd.service -b --no-pager -n 200
```

forensics가 배포된 host에서는 자동 snapshot이 생성됐는지 먼저 확인한다. 자동 trigger가 없었다면 local 상태만 읽는 수동 snapshot을 한 번 실행할 수 있다.

```
sudo systemctl start decs-nfs-forensics-snapshot.service
sudo cat /var/lib/decs-nfs-forensics/last_snapshot.state
```

이미 mount된 경로에 D-state caller가 있으면 **rpc-gssd** restart, forced unmount, 반복 remount를 먼저 실행하지 않는다. 증거를 보존한 뒤 storage·network 상태와 kernel wait path를 기준으로 복구 범위를 결정한다.

새 디버깅 문서 작성 형식

새 문제는 한 파일에 다음 내용을 남긴다.

1. **상태와 영향 범위**: 발생 시각, host, kernel, mount source/target/security

2. **증상**: 사용자 증상, process state, wait channel, alert
3. **확인한 사실과 가설**: 관측 사실과 아직 증명하지 못한 추론을 분리
4. **실험 목적**: 어떤 가설을 반증하거나 확인하려 했는지
5. **재현 조건**: server/client, version, cache, 동시성, fault와 안전장치
6. **실행 명령**: 사전 확인, fault 주입, workload, 관측, cleanup 순서
7. **결과와 판정 기준**: 성공/실패를 판단한 trace와 metric
8. **복구와 예방**: 임시 containment, 영구 수정, monitoring 변경
9. **원본 증거**: 결과 README, curated evidence, script와 checksum 링크

운영 환경에서 fault를 주입한 명령은 일반 runbook처럼 제시하지 않고 반드시 실행 일자와 승인·안전 조건을 붙인 **과거 실행 기록**으로 표시한다. 반복 실험은 가능하면 격리 VM harness로 옮긴다.

LAB9에서 **sec=krb5** 를 사용하는 이유

상태: LAB9에서 **sec=krb5p** 전환 전 24시간과 **sec=krb5** 전환 후 약 51시간을 비교한 운영 관측이다. 전환 뒤 NFS 지연과 적체가 크게 감소했지만, 관측 기간이 다르고 security flavor만 통제된 실험이 아니므로 **krb5p** 를 단일 원인으로 확정하지는 않는다.

1. 결론

LAB9의 Kerberos NFS 기본값은 **sec=krb5** 로 유지한다. 이 선택은 Kerberos principal 인증을 유지하면서 RPC payload 암호화 비용을 제외하고, LAB9에서 실제로 관측된 D-state 지속·SUNRPC 적체·mount probe 지연을 낮추기 위한 운영 기준이다.

krb5p 가 잘못된 방식이라는 뜻은 아니다. NFS payload 기밀성이 반드시 필요한 경로에서는 **krb5p** 를 사용해야 한다. 다만 현재 LAB9 사용자 공유의 기본 profile은 가용성과 지연 안정성을 우선해 **krb5** 를 사용한다. **krb5i** 또는 **krb5p** 는 요구사항이 있고 부하 검증을 통과한 경로에서만 명시적으로 선택한다.

2. security flavor의 차이

flavor	제공하는 보호	LAB9에서의 위치
sec=krb5	Kerberos principal 인증	기본값
sec=krb5i	인증 + RPC payload 무결성	변조 방지가 명시적으로 필요한 경로에서 검토
sec=krb5p	인증 + 무결성 + RPC payload 암호화	기밀성이 필요한 경로에서 성능 검증 후 사용

`sec=krb5`에서는 사용자와 NFS service가 Kerberos로 서로 인증되지만, NFS payload 자체는 암호화되지 않는다. 따라서 패킷 캡처와 forensics evidence는 root만 읽을 수 있게 보관하고, 보호되지 않은 네트워크에서 기밀 데이터를 전송해야 한다면 이번 성능 결과만으로 `krb5`를 선택해서는 안 된다.

3. 왜 전환했나

`sec=krb5p` 사용 중 LAB9에서는 NFS 관련 D-state가 관측 기간 내내 존재했고, SUNRPC task와 transport reply 대기가 누적됐다. mount responsiveness probe도 설정된 10초 timeout 상한에 도달했다. 이 상태에서는 home 경로를 읽는 SSH나 사용자 process가 NFS 대기에 연쇄적으로 묶일 수 있다.

`krb5p`는 각 RPC payload에 무결성 처리와 암호화를 추가한다. 이 비용이 장애의 단일 원인이라고 입증한 것은 아니지만, storage·kernel·network에서 이미 발생한 지연을 더 크게 보이게 하는 증폭 요인인지 확인할 운영상 이유가 있었다. 그래서 principal 인증은 유지하면서 privacy 처리를 제외하는 `sec=krb5`로 전환하고 동일한 NFS health 지표를 계속 관측했다.

4. 전환 전후 결과

측정 대상은 LAB9다.

지표	<code>krb5p</code> 전환 전 24시간	<code>krb5</code> 전환 후 약 51시간
NFS D-state 존재 비율	100%	약 4.1%
D-state 프로세스 최대	28개	8개
최장 연속 D-state	83,339초(약 23.1시간)	1,236초(약 20분 36초)
<code>xprt_pending</code> 평균	5.35	1.29
RPC task 최대	3,696	525
마운트 probe 최대	10초	0.429초
lease 만료	없음	없음

변화 폭은 다음과 같다.

비교 항목	변화
NFS D-state 존재 비율	95.9%p 감소
D-state 프로세스 최대	약 71.4% 감소
최장 연속 D-state	약 98.5% 감소
<code>xprt_pending</code> 평균	약 75.9% 감소
RPC task 최대	약 85.8% 감소

비교 항목	변화
마운트 probe 최대	약 95.7% 감소

가장 중요한 변화는 D-state가 “항상 존재하는 상태”에서 짧고 간헐적인 상태로 바뀐 점이다. `xprt_pending` 평균과 RPC task 최대도 함께 감소했으므로 단순히 D-state process 이름만 달라진 것이 아니라 NFS transport 적체 자체가 줄어든 방향과 일치한다.

전환 전 mount probe의 10초는 `cluster-monitor-exporter`의 command timeout 상한과 같다. 따라서 이 값은 정상 응답에 10초가 걸렸다가보다 probe가 timeout 또는 기존 in-flight probe 상태에 도달했을 가능성이 크다. 전환 후 최대 0.429초는 관측 구간에서 10초 상한에 도달하지 않았음을 보여준다.

두 구간 모두 lease 만료가 없었다. 따라서 이번 비교에서 확인한 개선은 lease 만료 복구 여부보다 D-state 지속, RPC queue와 mount 응답성에 관한 것이다.

5. 지표의 의미

지표	판정 방법	구현
NFS D-state 존재 비율	유효 sample 중 NFS/RPC 관련 D-state process가 하나 이상인 sample 비율	<code>cluster_monitor_nfs_forensics_d_state_processes</code>
D-state 프로세스 최대	관측 구간의 NFS/RPC 관련 D-state process 최대값	같은 metric의 <code>max_over_time</code>
최장 연속 D-state	가장 오래 연속 관측된 NFS/RPC 관련 D-state process의 시간	<code>cluster_monitor_nfs_forensics_d_state_oldest_seconds</code>
<code>xprt_pending</code> 평균	transport reply를 기다리는 SUNRPC task 수의 구간 평균	<code>cluster_monitor_nfs_xprt_pending</code>
RPC task 최대	NFS RPC client task 수의 구간 최대	<code>cluster_monitor_nfs_rpc_tasks</code>
마운트 probe 최대	required mount에 수행한 bounded <code>statfs</code> 시간의 구간 최대	<code>cluster_monitor_host_mount_probe_seconds</code>
lease 만료	local NFSv4 lease deadline을 넘긴 시간	<code>cluster_monitor_nfs_lease_expired_seconds</code>

수집 경로는 다음과 같다.

```
LAB9 kernel:/proc 상태
-> NFS forensics watcher status.json
-> cluster-monitor-exporter metrics
-> FARM Prometheus
-> 같은 server="LAB9" label의 전환 전후 구간 집계
```

구현은 `forensics watcher`, `forensics metric adapter`, `mount/D-state collector`에서 확인한다.

6. 이 결과로 말할 수 있는 것과 없는 것

말할 수 있는 범위는 다음과 같다.

- LAB9에서 **krb5** 전환 뒤 모든 주요 NFS 적체 지표가 같은 방향으로 개선됐다.
- 약 51시간의 전환 후 구간에는 24시간 전환 전 구간과 같은 장시간 고착이 관측되지 않았다.
- Kerberos 인증을 유지한 **krb5**가 LAB9의 현재 운영 baseline으로 더 안정적이었다.

아직 확정할 수 없는 범위는 다음과 같다.

- **krb5p**의 암호화 비용만으로 전환 전 D-state가 발생했다는 인과관계
- 같은 시기에 달라졌을 수 있는 workload, storage queue, network, kernel 상태의 영향
- FARM/LAB의 다른 host에서도 감소 폭이 동일할 것이라는 일반화
- 51시간보다 긴 기간이나 peak workload에서도 같은 결과가 유지된다는 보장

따라서 이 기록의 판정은 **LAB9 운영 선택을 지지하는 강한 전후 관측**이며, 통제된 benchmark나 원인 확정 실험은 아니다. 원인을 더 좁히려면 동일 workload와 동일 시간대에 **krb5** / **krb5p**를 반복 교차하는 격리 실험이 필요하다.

7. 운영 기준과 확인 방법

LAB9의 desired mount option은 **sec=krb5**다. mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 사용한다. D-state가 이미 존재할 때는 확인을 위해 mount를 반복해서 읽거나 즉시 remount하지 말고 먼저 local forensics 상태를 보존한다.

현재 mount의 security flavor는 다음처럼 정확히 확인한다.

```
LAB9_MOUNT_TARGET=/home/tako9/share

findmnt -T "$LAB9_MOUNT_TARGET" -n -o TARGET,SOURCE,FSTYPE,OPTIONS
findmnt -T "$LAB9_MOUNT_TARGET" -n -o OPTIONS \
| tr ',' '\n' \
| grep -Fx 'sec=krb5'
```

grep -F 'sec=krb5'만 사용하면 **sec=krb5p**도 일치하므로 option을 쉼표 단위로 분리해 exact match한다. 전환 이후에도 최소한 다음 조건을 함께 확인한다.

- NFS D-state 존재 비율과 최장 연속 시간이 다시 증가하지 않는가
- **xprt_pending** 평균과 RPC task 최대가 전환 전 수준으로 돌아가지 않는가
- mount probe가 10초 timeout 상한에 다시 도달하지 않는가
- **cluster_monitor_nfs_lease_expired_seconds**가 계속 0인가
- payload 기밀성 요구가 새로 생겨 **krb5p**가 필요한 경로가 되지 않았는가

기본 security policy는 Kerberos/NFS 운영 저장소, 배포 값은 **monitoring exporter** 변수를 기준으로 한다. 상세 장애 채증과 D-state 대응은 **디버깅 로그 개요**의 절차를 따른다.

8. 다음 측정에서 함께 남길 값

현재 비교표에는 구간 길이만 있고 정확한 시작·종료 timestamp와 원본 query/export artifact가 없다. 다음 재측정 부터는 아래 값을 함께 보관해야 같은 결과를 재계산할 수 있다.

- 전환 시각과 두 구간의 시작·종료 시각(타임존 포함)
- LAB9의 source FQDN, target, NFS version과 전체 mount option
- client/server kernel, storage 상태와 구간별 workload
- 사용한 PromQL 또는 `samples.tsv` 집계 script와 원본 결과
- exporter command timeout과 scrape interval
- 전환과 동시에 수행한 다른 설정 변경

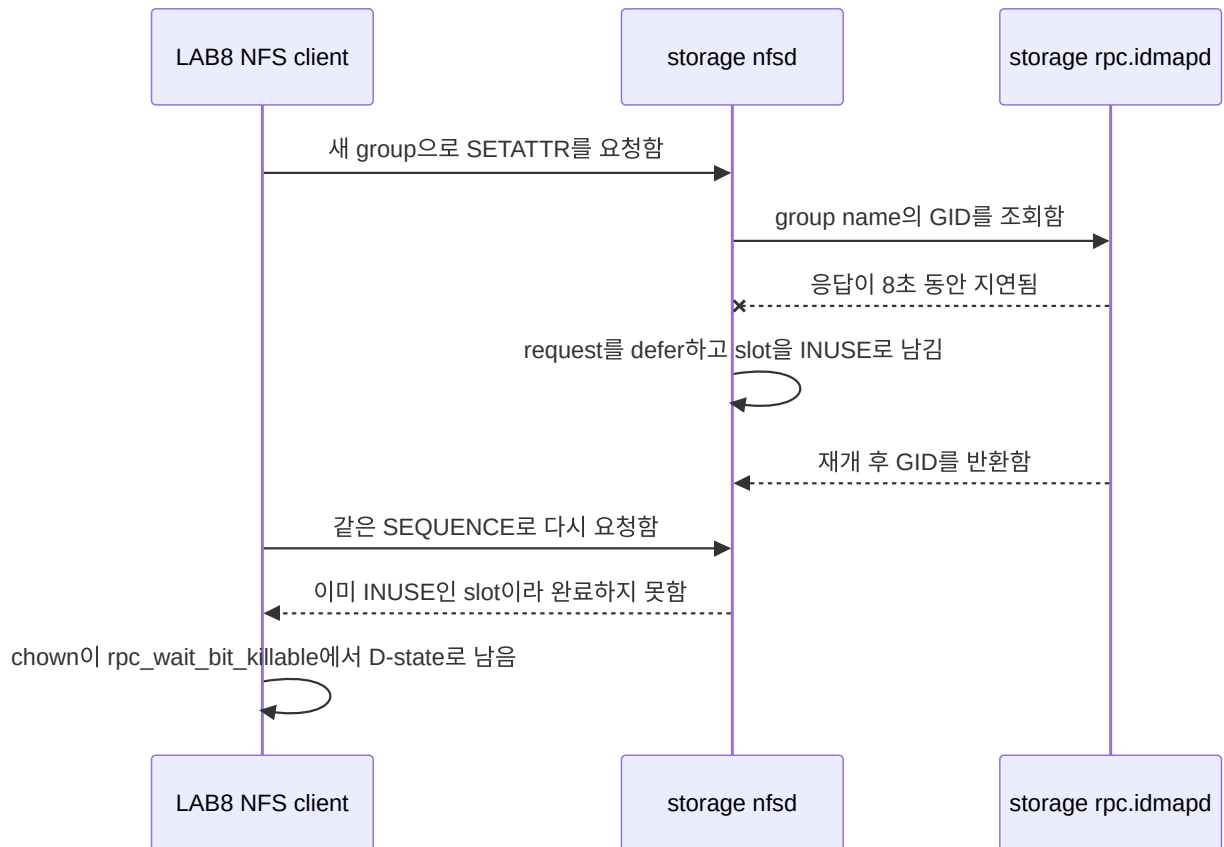
원본 artifact를 저장소에 추가하면 이 문서의 표 아래에 commit 고정 링크를 추가한다.

NFSv4.1 session slot 고착

상태: 2026-07-16에 slot leak 경로 재현 완료. 2026-06-29 LAB5 장애의 강한 원인 후보이지만, LAB5 장애 시작 시점의 storage trace가 없어 과거 장애의 최초 원인으로 확정하지는 않는다.

1. 요약

LAB storage의 구형 kernel `3.10.0-862.el7.x86_64` 에서 NFSv4 `SETATTR` 가 새로운 group 이름을 UID/GID로 해석하던 중 `rpc.idmapd` 응답이 지연되면 request가 defer될 수 있다. 이때 NFSv4.1 `SEQUENCE` 는 slot을 `INUSE` 로 표시하지만 response encode와 `nfsd4_sequence_done()` 이 실행되지 않아 slot이 반환되지 않는 경로를 실제로 재현했다.



한 slot만 leak한 최소 재현에서는 `ForeChannel` waiter가 0일 수도 있다. 따라서 waiter가 없다는 사실만으로 slot 문제를 배제하면 안 된다. session reset이나 recovery가 모든 slot의 drain을 기다리는 상황에서는 고착된 한 slot이 전체 복구를 막을 수 있다.

2. 어떤 증상이었나

2026-07-16 재현에서는 LAB8의 `chown` worker가 173초 이상 D-state에 남았다.

```

chown
-> nfs4_proc_setattr
-> nfs4_call_sync_sequence
-> rpc_wait_bit_killable
  
```

storage trace에서는 첫 처리와 재방문이 다음처럼 달랐다.

단계	관측
첫 idmap decode	<code>RQ_USEDEFERRAL</code> , <code>RQ_DROPME</code> 가 설정됨
첫 SEQUENCE 확인	<code>seqid=0x5f</code> , <code>slot_seqid=0x5e</code> , <code>INUSE=0</code>
첫 request 종료	dispatcher가 0을 반환했고 encode와 <code>sequence_done</code> 이 실행되지 않음
<code>rpc.idmapd</code> 재개 후	같은 request가 <code>seqid=0x5f</code> , <code>slot_seqid=0x5f</code> , <code>INUSE=1</code> 로 재방문

단계	관측
이후 retry	약 15초마다 같은 <code>INUSE=1</code> 을 확인하고 완료하지 못함

3. 왜 이 실험을 했나

조사는 다음 순서로 가설을 좁혔다.

1. 2026-06-29 LAB5에서는 NFS user home을 읽는 `sshd: ... [priv]` 가 D-state에 누적됐다.
2. 2026-07-10 LAB8에서 TCP/2049 transport를 일시 차단하자 같은 `rpc_wait_bit_killable` 계열 증상을 재현했지만, network가 복구된 뒤 process도 자연 복구됐다.
3. keytab, `rpc-gssd`, readiness와 mount 순서를 바로잡은 clean boot에서도 구형 storage kernel의 session state가 영구 고착될 수 있는지는 별도 검증이 필요했다.
4. upstream의 idmap deferral 수정 내용이 “NFSv4 decode 중 request drop으로 `NFSD4_SLOT_INUSE` 가 해제되지 않는다”는 경로를 설명하므로, 실제 LAB kernel과 mount에서 그 primitive를 최소 workload로 확인했다.

실험 목적은 많은 client를 동시에 멈추게 하는 것이 아니라, **idmap miss 하나가 slot 하나를 반환 불가능하게 만들 수 있는지**를 확인하는 것이었다.

4. 실험 결과

4.1 실제 LAB 환경의 최소 재현

첫 pass에서 `RQ_DROPME` 가 설정된 뒤 slot이 `INUSE` 로 바뀌었지만 encode와 `nfdsd4_sequence_done()` 이 호출되지 않았다. 8초 뒤 `rpc.idmapd` 가 재개된 후에도 같은 sequence는 이미 사용 중인 slot으로 판단됐다. client retry도 동일 상태를 반복했고 worker는 D-state에서 끝나지 않았다.

따라서 다음 좁은 결론은 확정할 수 있다.

- 당시 storage kernel에서는 idmap decode deferral이 NFSv4.1 session slot 하나를 leak할 수 있다.
- keytab 발급, `rpc-gssd` 시작 순서나 client mount 순서만 고쳐서는 이 server-side kernel 경로를 제거할 수 없다.
- 한 slot leak은 재현했지만 LAB5에서 나중에 관측한 전체 session recovery/drain stall까지 이 최소 실험 하나로 재현한 것은 아니다.
- LAB5 장애 시작 시점의 trace가 없으므로 어떤 실제 operation이 최초 idmap miss를 만들었는지는 알 수 없다.

4.2 A-G: 8초 idmap 지연 kernel matrix

최소 재현과 같은 `idmap-setattr`, NFSv4.1, `sec=krb5p` 조건에서 server와 client kernel 조합을 바꿔 각 5회 실행했다. client OS는 모두 Ubuntu 22.04.5 LTS이며, F/G만 client kernel을 `6.18.38` 로 올렸다.

Case	NFS server OS	Server kernel	Client kernel	결과
A	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5

Case	NFS server OS	Server kernel	Client kernel	결과
B	CentOS Linux 7.9.2009	3.10.0-1160.el7.x86_64	6.8.0-134-generic	slot 고착 5/5
C	CentOS Stream 10	6.12.74	6.8.0-134-generic	slot 고착 5/5
D	CentOS Stream 10	6.12.75	6.8.0-134-generic	자동 복구 5/5
E	CentOS Stream 10	6.12.0-248.el10.x86_64	6.8.0-134-generic	자동 복구 5/5
F	CentOS Linux 7.9.2009	3.10.0-862.el7.x86_64	6.18.38	slot 고착 5/5
G	CentOS Linux 7.9.2009	3.10.0-1160.el7.x86_64	6.18.38	slot 고착 5/5

구형 server kernel에서는 client kernel을 6.18.38 로 올린 F/G도 모두 고착됐다. 반대로 idmap/slot fix가 포함된 D와 signed Stream kernel E는 모두 자동 복구했다. 이 결과는 문제와 복구 여부를 가르는 주된 변수가 client가 아니라 **NFS server kernel의 idmap deferral 처리**임을 지지한다.

4.3 D/E: 120·300·600초 장시간 idmap 지연 추가 실험

8초 지연만으로는 수정된 kernel이 짧은 지연에서만 우연히 복구한 것인지 판단하기 어려웠다. 그래서 자동 복구한 D/E만 대상으로 idmap cache miss 응답을 120초, 300초, 600초까지 지연하고 각 조건을 3회씩 다시 실행했다.

Case	Server kernel	idmap 지연	원래 chgrp	신규 RPC	최종 snapshot D / FC / lease
D-120s	6.12.75-nfs-slot	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0*
D-300s	6.12.75-nfs-slot	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
D-600s	6.12.75-nfs-slot	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0
E-120s	6.12.0-248.el10.x86_64	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-300s	6.12.0-248.el10.x86_64	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-600s	6.12.0-248.el10.x86_64	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0

여기서 FC는 NFSv4.1 ForeChannel waiter 수다. 600초 조건의 EINVAL은 원래 chgrp 요청의 application 결과이며, slot 고착 판정과 분리해야 한다. 같은 run에서 idmap 지연을 해제한 뒤 실행한 신규 RPC는 모두 성공했고, 최종 snapshot의 D-state, ForeChannel waiter와 lease_expired도 모두 0이었다. 따라서 600초에서 원래 작업이 실패했더라도 **session slot은 반환됐고 후속 요청을 막지 않았다**.

* D-120s 2회차의 2초 sampler는 종료 직전 D-state 1을 한 번 기록했다. 그러나 직후 final snapshot은 D-state 0이었고, 신규 RPC 성공, ForeChannel 0, lease_expired=0, analyzer verdict=pass였다. 지속된 NFS slot 고착으로 판정하지 않았다.

5. 실제 재현 조건

항목	2026-07-16 조건
client	LAB8, 100.100.100.108
server	lab-storage.lab.decs.internal , 100.100.100.100
mount	/home/tako8/share , NFSv4.1, sec=krb5p , hard
storage kernel	3.10.0-862.el7.x86_64
client 시작 상태	D-state 0, NFS RPC task 0, readiness ready
test identity	LAB8 local group decs_idmap_repro_424242 , GID 424242
test object	/home/tako8/share/.decs-idmap-slot-repro-20260716
fault	storage rpc.idmapd 에 8초간 SIGSTOP
workload	test file에 GID 424242 를 지정하는 chown(2) 한 번
안전장치	storage-local 8초·20초 SIGCONT timer를 먼저 예약
금지한 복구	재현 후 client reboot, unmount, signal, service restart를 하지 않음

boot 순서도 별도로 확인했다. LAB8은 rpc-gssd active → Kerberos/NFS readiness 완료 → mount 순서였고, 기존 /etc/krb5.keytab 의 KVNO는 2였다. 따라서 이번 결과를 keytab 생성 누락이나 잘못된 mount 시작 순서로 설명할 수 없다.

6. 당시 실행한 명령

경고: 아래 명령은 2026-07-16의 승인된 실제 실행 기록이다. storage의 rpc.idmapd 를 멈추므로 일반 점검이나 운영 재현 절차로 다시 실행하면 안 된다. 재실험은 8절의 격리 VM harness를 사용한다. 특히 kprobe 인자 register는 당시 x86_64 kernel symbol에만 맞춘 값이라 다른 kernel에서 그대로 사용하지 않는다.

6.1 client 사전 확인과 test 값 준비

LAB8에서 D-state와 RPC task가 없는지 확인한 뒤, storage에 없던 group 이름을 client가 encode하도록 local group과 test file을 준비했다.

```

ps -eo stat= | awk '$1 ~ /^D/ {n++} END {print "d_state=" (n+0)}'
grep -c '^RPC task' /proc/net/rpc/nfs 2>/dev/null || true
findmnt -T /home/tako8/share -n -o TARGET,SOURCE,FSTYPE,OPTIONS

sudo groupadd -g 424242 decs_idmap_repro_424242

sudo bash -c '
p=/home/tako8/share/.decs-idmap-slot-repro-20260716
rm -f "$p"
touch "$p"
chmod 0666 "$p"
stat -c "path=%n uid=%u gid=%g mode=%a ino=%i" "$p"
'

```

fault 전에 GID 424241 을 사용한 정상 `chown` 을 실행해 같은 경로가 idmap 지연 없이는 encode와 `sequence_done` 까지 완료되는 것을 먼저 확인했다.

```

sudo python3 -c 'import os; p="/home/tako8/share/.decs-idmap-slot-repro-20260716"; print("before", os.stat(p).st_gid); os.chown(p, -1, 424241); print("after", os.stat(p).st_gid)'

```

6.2 storage trace 설정

당시 storage에서 idmap decode, slot 검사, dispatcher return, response encode와 slot 반환 여부를 한 trace instance에 기록했다.

```

T=/sys/kernel/debug/tracing
I="$T/instances/decs-idmap-slot-repro"
sudo mkdir -p "$I"

for event in uid gid check_slot dispatch dispatch_ret encode sequence sequence_done; do
    echo "-:decs_idmap/$event" | sudo tee -a "$T/kprobe_events" >/dev/null || true
done

echo 'p:decs_idmap/uid nfsd_map_name_to_uid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/gid nfsd_map_name_to_gid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/check_slot check_slot_seqid seqid=%di:u32 slot_seqid=%si:u32 slot_inuse=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/dispatch nfsd_dispatch rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'r:decs_idmap/dispatch_ret nfsd_dispatch ret=$retval:s32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/encode nfs4svc_encode_compoundres rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence nfsd4_sequence rqstp=%di cstate=%si seq=%dx' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence_done nfsd4_sequence_done resp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null

echo 0 | sudo tee "$I/tracing_on" >/dev/null
echo 0 | sudo tee "$I/events/enable" >/dev/null
sudo sh -c ": > '$I/trace'"
echo 8192 | sudo tee "$I/buffer_size_kb" >/dev/null

for event in \
    decs_idmap/uid decs_idmap/gid decs_idmap/check_slot decs_idmap/dispatch \
    decs_idmap/dispatch_ret decs_idmap/encode decs_idmap/sequence \
    decs_idmap/sequence_done sunrpc/svc_process sunrpc/svc_defer \
    sunrpc/svc_revisit_deferred sunrpc/svc_drop sunrpc/svc_send
do
    echo 1 | sudo tee "$I/events/$event/enable" >/dev/null
done
echo 1 | sudo tee "$I/tracing_on" >/dev/null

```

6.3 fault와 단일 workload 실행

storage에서 반드시 자동 재개 timer 두 개가 active인지 확인한 뒤 `rpc.idmapd`를 멈췄다.

```
pid="$(pgrep -xo rpc.idmapd)"

sudo systemd-run --unit=decs-idmapd-auto-resume-8s \
  --on-active=8s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"
sudo systemd-run --unit=decs-idmapd-auto-resume-20s \
  --on-active=20s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"

sudo systemctl is-active --quiet decs-idmapd-auto-resume-8s.timer
sudo systemctl is-active --quiet decs-idmapd-auto-resume-20s.timer
sudo kill -STOP "$pid"
```

그 직후 LAB8의 별도 systemd worker에서 `chown(2)` 한 번만 실행했다.

```
sudo systemd-run --unit=decs-idmap-slot-repro-worker --no-block \
  /usr/bin/python3 -c \
  "import os; os.chown('/home/tako8/share/.decs-idmap-slot-repro-20260716', -1, 424242)"
```

6.4 결과 확인과 trace 보존

새로운 NFS I/O를 만들지 않고 worker, local kernel state와 이미 동작 중이던 forensics state를 확인했다.

```
systemctl --no-pager --full status decs-idmap-slot-repro-worker.service
ps -eo pid=,ppid=,stat=,comm=,wchan:32=,args= \
  | awk '$3 ~ /^D/ {print}'
grep -E 'lease_expired|RPC task|xprt_pending|ForeChannel' \
  /proc/net/rpc/nfs 2>/dev/null || true

cat /run/decs-nfs-forensics/status.json
cat /var/lib/decs-nfs-gss/recovery.state
cat /var/lib/decs-nfs-gss/canary.state
```

storage trace는 다음 pattern을 기준으로 추렸다.

```
I=/sys/kernel/debug/tracing/instances/decs-idmap-slot-repro
sudo sh -c "echo 0 > '$I/tracing_on'"
sudo cp "$I/trace" /var/tmp/decs-idmap-slot-repro-20260716.trace

sudo grep -E \
  'decs_idmap_gid|svc_defer|svc_revisit_deferred|svc_drop:|check_slot|sequence_done|dispatch_ret' \
  /var/tmp/decs-idmap-slot-repro-20260716.trace
```

7. Monitoring과 운영 대응

재현 당시 guardrail은 의도대로 동작했다.

- forensics watcher가 `nfs_d_state` 이유로 incident snapshot을 보존했다.
- recovery는 `status=blocked`, `diagnosis=nfs_d_state_detected`, `retry_inhibited=1`로 바뀌었다.
- canary도 같은 진단으로 차단돼 추가 NFS request를 만들지 않았다.
- packet ring과 kernel trace는 계속 증거를 수집했다.

운영에서 같은 증상이 보이면 다음 순서를 따른다.

1. NFS path를 읽는 `find`, `du`, `stat`, canary를 추가 실행하지 않는다.
2. D-state stack, `/proc/net/rpc/nfs`, forensics status와 snapshot을 보존한다.
3. mount가 이미 있으면 자동 remount와 `rpc-gssd` 반복 restart를 차단한다.
4. storage kernel, `rpc.idmapd`, network와 NFS session 증거를 함께 확인한다.
5. 상태가 자연 복구되지 않으면 유지보수 창외 client reboot 또는 storage 조치를 수동 결정한다. reboot는 상태를 지울 뿐 원인 수정은 아니다.
6. 영구 조치는 idmap deferral fix가 포함된 storage kernel로 올리고 격리 회귀 실험을 통과시키는 것이다.

운영 mount를 `soft`로 바꾸는 방식은 timeout을 application error로 노출하고 data integrity 위험을 만들 수 있어 해결책으로 사용하지 않는다.

8. 격리 VM에서 다시 검증하는 방법

반복 실험은 LAB8 local disk의 `decs-nfs-slot-isolated` network에서 수행한다. 이 network는 forwarding이 없고 `100.100.100.0/24`, `192.168.1.0/24`와 운영 storage hostname을 거부한다. VM disk, trace와 result도 NFS share가 아니라 LAB8 local filesystem에 둔다.

kernel 비교 기준은 다음과 같다.

Case	Server kernel	Client kernel	목적
A	CentOS 7 3.10.0-862	Ubuntu 6.8.0-134	운영 취약 baseline
B	CentOS 7 3.10.0-1160	Ubuntu 6.8.0-134	CentOS 7 최종 kernel 비교

Case	Server kernel	Client kernel	목적
C	upstream 6.12.74	Ubuntu 6.8.0-134	fix 이전 positive control
D	upstream 6.12.75	Ubuntu 6.8.0-134	idmap/slot fix 비교 control
E	signed Stream 6.12.0-248.el10	Ubuntu 6.8.0-134	운영 후보
F	CentOS 7 3.10.0-862	kernel.org 6.18.38	새 client kernel 비교
G	CentOS 7 3.10.0-1160	kernel.org 6.18.38	새 client kernel 비교

준비와 격리는 VM lab README의 절차를 따른다.

```
cd /home/jy/server_manage/kerberos-nfs/test/lab-kerberos-poc/vm-slot-regression

sudo ./bin/provision_lab8_vms.sh "$PWD"
./bin/configure_vm_stack.sh
sudo ./bin/prefetch_kernel_matrix.sh
sudo ./bin/verify_kernel_sources.sh
sudo ./bin/build_upstream_kernels_vm.sh
sudo ./bin/prepare_centos7_domains.sh "$PWD"
sudo ./bin/isolate_test_vms.sh
```

단일 비교는 같은 idmap-setattr, sec=krb5p, 8초 조건을 case별로 실행한다.

```
./bin/switch_kernel_case.sh A
NFS_SLOT_EXPECT_VULNERABLE=1 \
./bin/run_single_case.sh A krb5p idmap-setattr 8 1

./bin/switch_kernel_case.sh D
NFS_SLOT_EXPECT_VULNERABLE=0 \
./bin/run_single_case.sh D krb5p idmap-setattr 8 1
```

먼저 실행 수만 확인하려면 plan-only mode를 쓴다.

```
MATRIX_PLAN_ONLY=1 \
MATRIX_CASES='A C D E' \
MATRIX_SECURITY='krb5p' \
MATRIX_SCENARIOS='idmap-setattr' \
MATRIX_DURATIONS='8' \
MATRIX_REPETITIONS=3 \
./bin/run_matrix.sh
```

취약 case가 고착되면 harness는 evidence를 archive한 뒤 **격리 VM만** reset한다. 그 reset은 자동 복구 성공으로 계산하지 않는다. 운영 LAB8 host와 운영 NFS mount는 이 harness의 복구 대상이 아니다.

9. 원본 증거와 코드

- 2026-07-16 실제 재현 결과
- curated trace evidence
- 2026-07-10 NFS transport fault 실험
- 격리 VM slot regression lab
- single-case harness
- result analyzer
- upstream stable patch 설명
- NFS forensics 구현

보존된 storage raw trace의 SHA-256은 `5fe7fbab2b8aa712ff71afe5156438b4e880edcc39cf16a00592e22d578bac59` 다.

NFSv4.1 session slot 고착 해결 및 운영 적용

상태: 2026-07-21~22 LAB storage의 OS와 kernel을 교체하고 Kerberos/NFS 구성을 복원했다. 2026-07-23 LAB8 운영 mount에서 `rpc.idmapd` 를 15초 지연한 직접 시험에서도 session slot이 영구 고착되지 않았다.

이 문서는 **NFSv4.1 session slot 고착**에서 확인한 server-side kernel 문제를 실제 LAB storage에서 어떻게 제거하고 운영 구성을 복원했는지 기록한다. 원인 재현 과정과 fault-injection 명령은 기존 문서에 두고, 여기에는 해결 조치와 안전한 사후 검증만 정리한다.

1. 해결 결론

이 장애의 근본 조치는 **NFS server인 lab-storage**의 kernel을 `idmap deferral` 수정이 포함된 kernel 계열로 교체하는 것이다. client 재부팅, NFS remount 또는 `rpc-gssd` 재시작은 이미 고착된 상태를 지울 수는 있지만 server-side slot leak 경로를 없애지 못한다.

항목	변경 전	변경 후
작업 서버	<code>lab-storage.lab.decs.internal</code>	동일
OS	CentOS Linux 7	CentOS Stream 10
kernel	<code>3.10.0-862.el7.x86_64</code>	<code>6.12.0-250.el10.x86_64</code>
데이터	<code>/294t</code> XFS	기존 <code>/dev/sda1</code> 을 포맷하지 않고 <code>/294t</code> 에 재연결

항목	변경 전	변경 후
AD/KDC	LAB2 Samba AD	동일, storage keytab은 LAB2에서 재생성
NFS 보안	Kerberos NFS	SSSD·gssproxy·NFS export를 복원

격리 VM 비교에서는 upstream 6.12.74 까지 slot 고착이 발생했고 6.12.75 에서 자동 복구했다. CentOS Stream 10 의 signed kernel 6.12.0-248.el10 도 같은 시험을 통과했다. 현재 운영 kernel 6.12.0-250.el10 은 그보다 뒤에 설치한 같은 배포판 kernel이다.

여기서 250.el10 은 upstream Linux 6.12.250 을 뜻하지 않고 CentOS/RHEL 계열의 package release 번호다. 초기 운영 적용은 격리 VM 회귀 결과와 교체 후 실제 서비스 상태를 근거로 판정했고, 2026-07-23에는 승인된 15초 지연 시험을 LAB8 운영 mount에서 한 번 추가해 현재 kernel에서도 영구 slot 고착이 없음을 확인했다.

2. 서버별 역할

서버	이번 조치에서의 역할
lab-storage.lab.decs.internal (100.100.100.100)	kernel 문제를 제거한 NFS server. OS, network, data mount, SSSD, keytab, gssproxy와 export를 복원한 대상
LAB2 / 100.100.100.102	운영 Samba AD DC, DNS/KDC. storage machine account와 NFS service principal의 원본
LAB1~LAB10	NFS client. 각 서버의 기존 /etc/fstab 으로 /home/takoN/share 를 다시 mount
LAB8 host	/home/tako8/share 운영 mount에서 15초 idmap 지연을 직접 검증한 client
LAB8의 격리 VM	kernel matrix와 장시간 지연 회귀 시험 전용

keytab은 예전 파일을 backup해서 되돌리지 않았다. LAB2 AD에 등록된 storage machine account와 SPN을 기준으로 새 keytab을 만든 뒤 lab-storage 의 /etc/krb5.keytab 에 설치했다. 따라서 keytab 생성 작업은 LAB2에서, 설치와 검증은 lab-storage 에서 수행한다.

3. 실제 적용 내용

3.1 lab-storage : OS와 kernel 교체

2026-07-21 22:38 KST에 kernel-core-6.12.0-250.el10.x86_64 를 설치했고 22:49 KST부터 이 kernel로 부팅해 운영 중이다. hostname과 두 network를 다음 상태로 복원했다.

용도	interface	주소
관리망	enp24s0f1	192.168.1.20/24
storage망	enp101s0f0	100.100.100.100/24

hostname은 `lab-storage.lab.decs.internal`, timezone은 `Asia/Seoul`이며 NTP 동기화 상태는 `yes`다.

3.2 lab-storage : 기존 데이터 디스크 재연결

RAID volume이 처음 보이지 않은 이유

OS 설치 직후에는 294TB 데이터 RAID가 정상적인 `/dev/sda`로 나타나지 않았다. 이 장비의 controller는 다음 Areca ARC-1284다.

```
17:00.0 RAID bus controller: Areca Technology Corp. ARC-12x4
Subsystem: Areca Technology Corp. ARC-1284 24 Port
PCI ID: 17d3:1214
```

CentOS Stream 10의 운영 kernel 설정을 확인하면 ARC-1284용 `arcmsr`가 빠져 있다.

```
grep CONFIG_SCSI_ARCMSR /boot/config-"$(uname -r)"
```

당시 결과는 다음과 같았다.

```
# CONFIG_SCSI_ARCMSR is not set
```

먼저 `kernel-modules-extra`를 설치했지만 이 kernel용 `arcmsr.ko`는 들어 있지 않았다.

```
sudo dnf install -y kernel-modules-extra
modinfo arcmsr
```

ELRepo의 `kmod-arcmsr`도 확인했지만 EL10용 package가 없었다. 최종 해결은 ELRepo package 설치가 아니라 **현재 실행 kernel에 맞춰 `arcmsr`를 외부 module로 직접 빌드하는 것이었다.**

`arcmsr` 외부 module 빌드와 설치

2026-07-21에 실제로 설치한 build dependency는 다음과 같다. `kernel-devel`의 version은 반드시 module을 사용할 kernel과 정확히 같아야 한다.

```
KVER=$(uname -r)

sudo dnf install -y \
    "kernel-devel-$KVER" \
    gcc make elfutils-libelf-devel xz
```

Linux 6.12의 `drivers/scsi/arcmsr`에서 `Makefile`, `arcmsr.h`, `arcmsr_attr.c`, `arcmsr_hba.c`를 임시 build directory로 가져왔다. Stream 10 kernel에 backport된 API에 맞게 다음 호환 조정을 적용한 뒤 외부 module로 빌드했다.

- `slave_configure` 를 `sdev_configure` 와 `struct queue_limits` signature로 변경
- `bios_param` 의 `struct block_device` 를 `struct gendisk` 로 변경
- `del_timer_sync()` 를 `timer_delete_sync()` 로 변경
- `from_timer()` 를 `timer_container_of()` 로 변경
- `sysfs bin_attribute` 인자를 `const` 로 변경
- PCI device ID table을 `const` 로 변경

실제 build 형태는 다음과 같다. upstream 6.12 source를 수정하지 않고 그대로 빌드하면 Stream 10 kernel API와 맞지 않으므로 위 호환 조정이 필요하다.

```
KVER=$(uname -r)
BUILD_DIR=$(mktemp -d /tmp/arcmsr-build.XXXXXX)

# BUILD_DIR에 호환 조정을 마친 다음 파일을 준비한다.
# Makefile arcmr.h arcmr_attr.c arcmr_hba.c

make -C "/usr/src/kernels/$KVER" \
    M="$BUILD_DIR" \
    CONFIG_SCSI_ARCMSR=m \
    modules

modinfo "$BUILD_DIR/arcmsr.ko"
```

먼저 `insmod` 로 현재 boot에서 controller와 RAID volume이 정상적으로 인식되는지 확인했다.

```
sudo insmod "$BUILD_DIR/arcmsr.ko"

lspci -nnk -s 17:00.0
lsblk -e7 -o NAME,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS,MODEL
```

`ARC-1284-VOL#000` 과 partition UUID가 확인된 후 module을 영구 경로에 설치하고 boot 때 자동으로 load되도록 등록했다.

```
KVER=$(uname -r)

sudo install -D -m 0644 "$BUILD_DIR/arcmsr.ko" \
    "/lib/modules/$KVER/extra/arcmsr.ko"
sudo depmod -a "$KVER"

printf 'arcmsr\n' | sudo tee /etc/modules-load.d/arcmsr.conf >/dev/null
```

현재 설치된 module은 다음 상태다.

```
path=/lib/modules/6.12.0-250.el10.x86_64/extra/arcmsr.ko
version=v1.51.00.14-20230915
sha256=8c78e3bb666abc49a17419904ab5911e0fed0c9cb885f0e2c18333b118d7865e
driver=arcmsr
volume=ARC-1284-VOL#000
```

이 module은 RPM 소유 파일이 아니며 서명되지 않은 외부 module이다. 현재 server는 Secure Boot가 disabled라 load됐고 kernel log에는 out-of-tree module taint가 기록된다.

kernel 업데이트 주의: 새 kernel을 설치하면 그 version에 맞는 `kernel-devel` 로 `arcmsr.ko` 를 다시 빌드해 새 kernel의 `/lib/modules/<새-version>/extra/` 에 설치하고 `depmod` 를 실행해야 한다. 이를 준비하지 않고 새 kernel로 reboot하면 ARC-1284 volume이 나타나지 않아 `/294t` mount가 실패할 수 있다.

XFS mount 복원

driver load 후 RAID volume은 `/dev/sda` , 기존 XFS partition은 `/dev/sda1` 로 나타났다. 데이터 디스크를 포맷하지 않고 다음 fstab 항목으로 다시 연결했다.

```
UUID=2a417935-7537-4301-bbcf-5ab2ca836af5 /294t xfs defaults 1 2
```

2026-07-22 확인 결과는 `/dev/sda1` → `/294t` , XFS, `rw` 다. 이 조치에서는 `/294t` 를 다시 포맷하지 않았다.

```
lsblk -e7 -o NAME,SIZE,FSTYPE,UUID,MOUNTPOINTS,MODEL
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS /294t
lspci -nnk -s 17:00.0
lsmod | grep '^arcmsr'
```

2026-07-22 이후 kernel log에는 일부 allocation group의 AGFL을 reset했고 block이 leak됐다는 XFS warning도 기록됐다. 이는 `arcmsr` load와 `/294t` mount 성공 여부와 별개인 filesystem metadata 점검 항목이다. 운영 export가 연결된 상태에서 `xfs_repair` 를 실행하지 말고, 별도 유지보수 창에 NFS를 중단하고 `/294t` 를 unmount한 뒤 offline 점검·복구해야 한다.

3.3 LAB2와 lab-storage : AD 설정과 SSSD·keytab 복원

LAB2: AD server와 storage principal 준비

LAB2는 `LAB.DECS.INTERNAL` realm의 Samba AD DC와 KDC 역할을 한다. storage망 주소는 `100.100.100.102` 이며, `lab-storage` 의 Kerberos 설정에서 KDC와 admin server로 사용한다.

LAB2 AD에서 `lab-storage` machine account와 NFS service principal을 확인한 뒤 keytab을 새로 생성했다. OS 재설치 전 keytab을 backup해서 복원하지 않고 AD를 원본으로 재발급한 것이다.

```
LAB-STORAGE$@LAB.DECS.INTERNAL
nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
```

생성된 keytab의 machine principal과 NFS service principal KVNO는 모두 2였다.

lab-storage : SSSD와 keytab 설치·검증

lab-storage 는 Kerberos realm을 LAB.DECS.INTERNAL , KDC와 admin server를 LAB2의 100.100.100.102 로 설정했다. SSSD는 다음 정책으로 AD의 RFC2307 UID/GID를 그대로 조회한다.

```
[domain/lab.decs.internal]
ad_domain = lab.decs.internal
krb5_realm = LAB.DECS.INTERNAL
id_provider = ad
access_provider = permit
ldap_id_mapping = False
use_fully_qualified_names = False
```

LAB2에서 재생성한 keytab은 lab-storage 의 /etc/krb5.keytab 에 root 전용 권한으로 설치했다.

lab-storage 에서 machine principal로 TGT를 받은 뒤 NFS principal을 keytab과 대조한 결과는 다음과 같다.

```
nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL: kvno = 2, keytab entry valid
```

NFS service principal 자체를 client principal처럼 kinit -k 하는 것은 이 구성의 검증 방법이 아니다. 아래 5.1절처럼 machine principal로 먼저 kinit 한 뒤 kvno -k 를 사용한다.

3.4 lab-storage : NFS service와 export 복원

nfs-server , sssd , gssproxy 를 활성화하고 NFSv4 port 2049/tcp 와 관련 firewall service를 복원했다. 2026-07-22 확인 시 nfsd thread는 16개였고 nfs-server , sssd , gssproxy 가 모두 active였다.

Kerberos 대상 export는 다음 상태다.

경로	client	security flavor
/294t/dcloud/share	100.100.100.101 ~ 110	krb5p:krb5i:krb5
/294t/share/test-krb	지정 시험 client	krb5p
/294t/health/nfs-gss-canary	100.100.100.101 ~ 110	krb5 , read-only

기존 sec=sys 전용 경로는 별도 호환 export로 유지했다. Kerberos export가 정상이라는 이유로 기존 sec=sys export를 임의로 삭제하지 않는다.

3.5 LAB1~LAB10: 기존 fstab mount 복원

client별 mount 설정을 새로 생성하지 않고 각 서버에 이미 있던 `/etc/fstab` 을 사용했다. mount가 없는 client에서 만 `sudo mount -a` 를 실행해 `/home/takoN/share` 를 다시 연결했다.

2026-07-22 재점검에서 LAB1~LAB10 모두 다음 공통 상태를 보였다.

```
source=lab-storage.lab.decs.internal:/294t/dcloud/share
fstype=nfs4
vers=4.1
hard
sec=krb5
addr=100.100.100.100
```

최초 병렬 점검 중 LAB7에서 짧은 D-state process 1개가 관측됐지만 즉시 수행한 local 상태 재확인에서는 사라졌다. 같은 시점에 NFS RPC task, transport pending, ForeChannel waiter와 NFS D-state가 모두 0이었고 recovery 상태도 `healthy` 였다. 이를 지속된 session slot 고착으로 판정하지 않았다.

3.6 LAB8 운영 mount: 15초 `rpc.idmapd` 지연 시험

과거 실행 기록: 아래 내용은 안전장치를 준비한 뒤 한 번 수행한 운영 검증 결과다. 일반 점검 절차가 아니며 이 문서만 보고 운영 storage에서 반복 실행하지 않는다.

격리 VM 결과와 별도로 현재 운영 중인 storage kernel에서 같은 idmap 지연 경로가 영구 slot 고착을 만들지 않는지 직접 확인했다.

항목	시험 조건
NFS client	LAB8 host의 운영 mount <code>/home/tako8/share</code>
mount	NFSv4.1, <code>sec=krb5</code> , <code>hard</code>
NFS server	<code>lab-storage.lab.decs.internal</code>
storage kernel	<code>6.12.0-250.el10.x86_64</code>
workload	시험 container 내부에서 새 GID <code>424242</code> 로 test file에 <code>chown</code>
idmap fault	storage의 <code>rpc.idmapd</code> 응답을 15초 지연
안전장치	15초 자동 재개 timer와 독립적인 30초 이중 자동 재개 timer

지연 중에는 `chown` 이 `rpc_wait_bit_killable` 경로에서 일시적으로 D-state에 들어갔다. 15초 자동 재개 후 worker는 더 이상 D-state에 남지 않고 종료됐다. 원래 `chown` 의 application 결과는 `EINVAL` 이었다.

```
/bin/chown: changing group of '.../chown-target': Invalid argument
```


이 `EINVAL` 은 “`chown` 성공”을 의미하지 않는다. 하지만 취약 kernel에서처럼 동일 요청이 `INUSE` slot에 계속 걸려 영구 D-state로 남지도 않았다. 이후 test file 삭제 RPC가 성공했고 client reboot, unmount, NFS service restart 없이 정상 상태로 돌아왔다. 후속 확인 결과는 다음과 같았다.

판정 항목	결과
지속 NFS D-state	0
NFS RPC task	0
transport pending	0
ForeChannel waiter	0
<code>lease_expired</code>	0
후속 NFS 작업	test file 삭제 성공

따라서 이 시험은 운영 kernel `6.12.0-250.el10` 에서 15초 idmap 지연이 일시적인 RPC 대기를 만들 수는 있지만 **NFSv4.1 session slot**을 영구 고착시키지는 않았음을 보여준다. 원래 operation의 application 결과와 session recovery 판정은 분리해야 한다.

4. 해결 판정 기준

이번 조치는 다음을 모두 만족해 운영 적용 완료로 판정했다.

- `lab-storage` 가 취약한 `3.10.0-862.el7` 이 아니라 `6.12.0-250.el10` 으로 부팅돼 있다.
- `/294t` 가 기존 XFS data disk에서 `rw` 로 mount돼 있다.
- LAB2 machine principal의 TGT 발급과 NFS service principal의 keytab 검증이 성공한다.
- SSSD에서 AD 사용자의 RFC2307 UID/GID가 조회된다.
- `nfs-server` , `sssd` , `gssproxy` 가 active이고 NFSv4 port가 listen한다.
- Kerberos NFS export가 LAB1~LAB10 storage망 주소에 열려 있다.
- LAB1~LAB10에서 기존 fstab의 NFSv4.1 `sec=krb5` mount가 확인된다.
- 지속되는 NFS D-state, RPC task, ForeChannel waiter 또는 lease expiry가 없다.
- LAB8 운영 mount의 15초 idmap 지연 후에도 workload가 영구 D-state에 남지 않고 후속 NFS RPC가 성공한다.

이 판정은 “2026-06-29 LAB5 장애의 최초 원인이 반드시 이 bug였다”는 뜻은 아니다. 당시 시작 시점의 storage trace가 없어 과거 장애 원인은 여전히 강한 후보로 남는다. 이번에 확정한 것은 구형 storage kernel에 실제 slot leak 경로가 있었고, 그 kernel을 운영에서 제거했다는 사실이다.

5. 안전한 사후 검증

5.1 작업 서버: `lab-storage`

```
hostname -f
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS /294t

systemctl is-active nfs-server sssd gssproxy
sudo exportfs -v
sudo ss -lnt | grep ':2049 '

DECS_KRB5CC_DIR=$(mktemp -d /tmp/decs-storage-kvno.XXXXXX)
DECS_KRB5CC="FILE:$DECS_KRB5CC_DIR/ccache"

sudo env KRB5CCNAME="$DECS_KRB5CC" \
  kinit -k -t /etc/krb5.keytab 'LAB-STORAGE$@LAB.DECS.INTERNAL'
sudo env KRB5CCNAME="$DECS_KRB5CC" \
  kvno -k /etc/krb5.keytab \
  nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
sudo env KRB5CCNAME="$DECS_KRB5CC" kdestroy
rmdir "$DECS_KRB5CC_DIR"
```

기대 kernel은 6.12.0-250.el10.x86_64 이상이며, kvno 결과에는 keytab entry valid 가 나와야 한다.

5.2 작업 서버: 각 NFS client LAB1~LAB10

아래의 N 은 현재 접속한 LAB 서버 번호로 바꾼다. 이미 mount돼 있으면 반복 remount하지 않는다. mount가 없고 D-state도 없을 때만 기존 fstab으로 mount한다.

```
ps -e -o state=,pid=,etimes=,comm=,wchan:48=,args= \
  | awk '$1 ~ /^D/ {print}'

findmnt -T /home/takoN/share -o TARGET,SOURCE,FSTYPE,OPTIONS

# findmnt에 결과가 없고 D-state process도 없을 때만 실행
sudo mount -a
findmnt -T /home/takoN/share -o TARGET,SOURCE,FSTYPE,OPTIONS
```

container의 실제 사용자 권한까지 확인할 때는 container 안에서 해당 사용자로 실행한다. 다른 사용자의 홈이나 파일에 임의로 chown 하지 않는다.

```
id
ls -al "$HOME/share" | sed -n '1,40p'

probe="$HOME/share/.nfs-post-upgrade-check.${hostname -s}.${$}"
printf 'nfs post-upgrade check\n' >"$probe"
grep -Fx 'nfs post-upgrade check' "$probe"
ls -ln "$probe"
rm -f "$probe"
```

`ls -al`의 소유자·그룹이 `nobody`가 아니라 AD의 사용자·그룹 이름으로 보이고, 생성·읽기·삭제가 모두 성공해야 한다.

6. 같은 증상이 다시 보일 때

NFS `hard` mount의 process가 D-state에 들어가면 NFS 경로를 읽는 `find`, `du`, `stat`, 반복 canary를 추가 실행하지 않는다. 새로운 NFS request와 D-state process를 더 만들 수 있다.

1. 먼저 client에서 `uname -r`, D-state stack, `/proc/net/rpc/nfs`, forensics와 recovery state를 local filesystem에서 수집한다.
2. `lab-storage`가 실수로 구형 kernel로 부팅되지 않았는지 확인한다.
3. 자동 remount, 강제 unmount, `rpc-gssd` 반복 restart를 중단한다.
4. storage의 network, nfsd, gssproxy와 session trace를 함께 확인한다.
5. 2026-07-23의 승인된 단일 시험을 일반 점검처럼 반복하지 않는다. 추가 재실험은 LAB8의 격리 VM harness에서 수행한다.
6. client reboot는 고착된 client state를 지우는 복구 수단일 뿐 근본 해결로 기록하지 않는다.

운영 mount를 `soft`로 바꾸는 것도 해결책으로 사용하지 않는다. timeout을 application error로 노출하고 data integrity 위험을 만들 수 있다.

7. 증거와 관련 문서

- 원인 재현과 kernel matrix
- 실제 LAB8 재현 결과
- 격리 VM 회귀 시험 절차
- LAB storage OS 재설치 후 복원 절차